

Switch as an expression :
From JDK 14 onwards, A switch statement can also **return some value** that means we can use switch statement as **switch expression**.

This is known as switch case with return value OR expression style switch.

Rules:

- a) While writing switch body ; (semicolon is compulsory at the end)
- b) Here default -> is compulsory [If no case will match then default will return value]
- c) From the switch body, we need to return appropriate value based on type.
- d) If, we have multiple statements ({ } using block) then yield keyword is compulsory to return the value explicitly
[In a single statement, value will return automatically]

SwitchExpression.java

```
-----
//Even number OR Odd number

void main()
{
    int number = Integer.parseInt(IO.readLine("Enter a number from 1 - 10 :"));

    String result = switch(number)
    {
        case 1,3,5,7,9 ->
        {
            String y = "Odd Number";
            yield y;
        }

        case 2,4,6,8,10 -> "Even Number";

        default -> "Invalid Input";
    };

    IO.println(number+" is :"+result);
}
```

SwitchDemo5.java

```
-----
//Print the message based on the student grade
void main()
{
    String grade = IO.readLine("Enter Student grade[A/B/C/D] :").toUpperCase();

    String result = switch(grade)
    {
        case "A", "B" ->
        {
            yield "Excellent!!!";
        }

        case "C" -> "Very good!!!";

        case "D" -> "Good!!!";

        default -> "Invalid Grade";
    };

    IO.println("Your grade is :"+grade+" and your result is "+result);
}
```

Note : return keyword is used to return a value and terminate the flow from **a method** but switch is **not a method** (does not contain return type) It is **an expression so, yield keyword is compulsory for block of statements**

SwitchDemo6.java

```
-----
/* Employee performer of the year
Grade A -> High-level performer [bonus amount is 20% of the salary]
Grade B -> Mid-level performer [bonus amount is 15% of the salary]
Grade C -> Basic level performer [bonus amount is 10% of the salary]
No grade -> 5 % of the salary
*/
void main()
{
    double salary = Double.parseDouble(IO.readLine("Enter your Salary :"));
    char grade = IO.readLine("Enter your grade :").toUpperCase().charAt(0);

    double bonus = switch(grade)
    {
        case 'A' ->
        {
            IO.println("High-level performer");
            yield salary * 0.20;
        }

        case 'B' ->
        {
            IO.println("Mid-level performer");
            yield salary * 0.15;
        }

        case 'C' ->
        {
            IO.println("Basic-level performer");
            yield salary * 0.10;
        }

        default -> salary * 0.05;
    };

    IO.println("Your Salary is :"+salary);
    IO.println("Your bonus amount is :"+bonus);
}
```

SwitchDemo7.java

```
-----
void main()
{
    long x = 12;

    switch(x)
    {
        case 12L:
    }
}
```

Note : We cannot pass long, float, double and boolean as well as respective Wrapper classes.

SwitchDemo8.java

```
-----
void main()
{
    int x = 11;
    final int y = 10;

    switch(x)
    {
        case y -> IO.println("It is case 10"); //y must be declared with final

        case 5 + 6 -> IO.println("It is case 11"); //Constant expression is valid
    }
}
```

* While working with byte OR short the label of switc must be within the range only otherwise we will get compilation error

```
byte range : -128 to 127
short range : -32768 to 32767
```

```
void main()
{
    byte b = 125;

    switch(b)
    {
        case 125 -> IO.println("Case 125");
        case 127 -> IO.println("Case 127");
        case 130 -> IO.println("Case 130"); //Invalid [error]
    }
}
```

```
void main()
{
    short b = 125;

    switch(b)
    {
        case 125 -> IO.println("Case 125");
        case 127 -> IO.println("Case 127");
        case 33000 -> IO.println("Case 130"); //Invalid
    }
}
```

//Based on the day input print Weekend OR weekday

```
void main()
{
    String day = IO.readLine("Enter the day :");

    String inputType = day.toUpperCase();

    String dayType = switch(inputType)
    {
        case "SAT", "SUN" -> "Weekend";

        default -> "Weekday";
    };

    IO.println("Today is :"+day+" and It is "+dayType);
}
```

Loops in Java :

If we want to execute **the same code** multiple times then writing the same code again and again is not a good choice.

Example:
I want to print "Welcome to Java" 10 times then writing this print statement 10 times is not a good programming practice.

Repeat.java

```
-----
void main()
{
    IO.println("Welcome to Java");
    IO.println("Welcome to Java");
    IO.println("Welcome to Java");
    IO.println("Welcome to Java");
    IO.println("Welcome to Java");
    IO.println("Welcome to Java");
    IO.println("Welcome to Java");
    IO.println("Welcome to Java");
    IO.println("Welcome to Java");
    IO.println("Welcome to Java");
}
```

Here we should use looping statement:

Loops in Java are used to execute a **block of code multiple times** as long as the given Condition is **true**. It also comes under control flow statement.

It helps us to reduce code duplication as well as to write the logic **once** and control how many times it runs using a condition.

Types of Loops in Java:

Java provides four types of Loops:

- 1) do while loop
- 2) while loop
- 3) for loop
- 4) for-each loop

Common Characteristics of loop:

- a) Initialization (Starting point of the loop)
- b) Condition (Ending point of the loop)
- c) Updating (Increment/Decrement)

1) do-while loop:

It is an **exit-controlled loop** because without checking the condition, it will execute the body at-least once.

Syntax:

```
do
{
    // statements
}while(condition);
```

DoWhileDemo1.java

```
-----
void main()
{
    do
    {
        int x = 1;
        IO.println(x);
        x++;
    }
    while (x<=10); //error [cannot find symbol x]
}
```

DoWhileDemo2.java

```
-----
void main()
{
    int x = 1;
    do
    {
        IO.println(x);
        x++;
    }
    while (x<=10);
}
```

DoWhileDemo3.java

```
-----
void main()
{
    int x = 1000;
    do
    {
        IO.println(x);
        x++;
    }
    while (x<=10);
}
```

Exit control behaviour

DoWhileDemo4.java

```
-----
void main()
{
    int x = 0; //x = 0

    do
    {
        IO.println(x);
        x += x++; // x = x + x++; //x = 0 + 0
    }
    while (x<=3);
}
```

Output : Infinite times 0

```
void main()
{
    int a = 5; //a = 6

    a += a++; //a = a + a++; a = 5 + 5

    IO.println(a); //10
}
```